

graPyte - przenośna biblioteka grafowa

Franciszek Moszczuk

April 2026

Contents

1	Wstęp	1
2	Rozwój biblioteki	2
3	Funkcjonalność	3
3.1	Lista Sąsiedztwa	3
3.1.1	Wydrukowanie listy sąsiedztwa.	3
3.1.2	Konwersja z/do Macierzy sąsiedztwa	4
3.2	Macierz Sąsiedztwa	4
3.2.1	Wydrukowanie informacji o macierzy	4
3.2.2	Konwersja z/do Listy sąsiedztwa	5
3.2.3	Wczytywanie grafu o formacie Graph6/Digraph6	5
3.2.4	N-Marszruty	5
3.2.5	Cykle długości 3	5
3.2.6	Suma krawędzi	6
3.2.7	Stopień wierzchołka	6
3.2.8	Liczba Annihilacji	6
4	Możliwości Rozwoju projektu	6
5	Wnioski oraz spostrzeżenia	7

1. Wstęp

graPyte (czytaj: *gra-pajt* lub *gra-bajt*) to prosta biblioteka grafowa opracowana w ramach laboratoriów z teorii grafów. Wspiera grafy skierowane i nieskierowane, listy wierzchołków, listy oraz macierze sąsiedztwa, czytanie formatu graph6 i digraph6¹, oraz wykonywanie prostych obliczeń oraz operacji na grafach (np. obliczenie stopnia wierzchołka, ilość cykli o długości 3, ilość N-marszrut dla danej krawędzi itp). Główną zaletą biblioteki jest mały zakres

¹graph6: <https://users.cecs.anu.edu.au/~bdm/data/formats.txt>

wymaganych zależności (wystarczy posiadać bibliotekę `numpy`, oraz standardową instalację Pythona w wersji 3.7+.), jak i przyjazny interfejs. Stworzenie nowego grafu celowo przypomina jego zapis formalny. Przykładowo, dla poniższego grafu:

$$G = (\{a, b, c, d\}, \{a, b\}, \{a, c\}, \{a, d\}, \{b, c\}, \{c, d\}) \quad (1)$$

...stworzenie na podstawie tego zapisu Macierzy sąsiedztwa jest następujące:

```

1 >>>from grapyte import *
2 >>>G = GraphAdj({'a', 'b', 'c', 'd'},
3 ...    [ ('a', 'b'), ('a', 'c'), ('a', 'd'),
4 ...    ('b', 'c'), ('c', 'd') ],
5 ...    name = "Lecture", directed=False)
6 >>> print(G)
7 Undirected Matrix Multigraph Lecture with the following params:
8 Vertex: {0: 'a', 1: 'b', 2: 'c', 3: 'd'}
9 Matrix:
10 a,      b,      c,      d
11 a [  0   1   1   1]
12 b [  1   0   1   0]
13 c [  1   1   0   1]
14 d [  1   0   1   0]
15 >>>M.count_3Cycles()
16 2

```

Obraz 1: Przykładowe użycie biblioteki `graPyte`.

2. Rozwój bilblioteki

Bibliotekę zaczęto od pisania klasy *GraphEdge*, która stanowiła szkielet do napisania reszty modułów. Postąpiono tak z racji błędnej interpretacji list sąsiedztwa, przez co implementacja listy była hybrydą pomiędzy dwoma rozwiązaniami, ale również z prostoty takiego rozwiązania. Oto jej uproszczony interfejs, bez opisu funkcji:

Jak widać na obrazku 2, każdy graf posiada (oprócz stworzenia grafu) możliwość dodawania/usuwania wierzchołków, przedstawienia go w konsoli oraz konwersji na inny typ. Argumenty funkcji wskazują na typ argumentu (poprzez zapis strzałkowy) potrzebny do wywołania konstruktora bądź metody klasy. Wszystkie klasy grafów tworzy się w ten sam sposób, z wyjątkiem wczytywania grafu z formatu `graph6/digraph6`. Dużym problemem było wybranie odpowiedniej struktury danych do każdej z klas, bowiem typ danych dyktuje dostępne operacje na obiekcie. Zależnie od wyboru może to utrudnić lub ułatwić późniejsze rozszerzanie tychże klas. Klasy **GraphEdge** oraz **GraphEdge** korzystają z klasy zbiorów (tak, by dało się zaimplementować operacje na zbiorach wierzchołków w późniejszych wersjach projektu poprzez przeładowania) oraz z odpowiednich listy i słownika mapującego nazwy wierzchołków do list krawędzi.

```

1 class GraphEdge:
2     """
3     Edge Multigraph class. Uses an edge list.
4     This was a starter template for other classes (like Graph
5     adjacency).
6
7     CC Franciszek Moszczuk
8     """
9     def __init__(self, vertex: set, edges: list, name = "Unnamed"):
10         ...
11     def __str__(self):
12         ...
13     def add_vertex(self, vertex: str):
14     def remove_vertex(self, vertex: str):
15     def add_edge(self, edge: tuple):
16     def remove_edge(self, edge: tuple):
17     def to_Graph(self):
18     def to_GraphAdj(self):

```

Obraz 2: Metody klasy GraphEdge.

Daje to też możliwość łatwego rozszerzać klas, przez co poprzez dziedziczenie można zaimplementować klasy grafów szczególnych i działających na specyficznej klasie algorytmów.

3. Funkcjonalność

Poza bazową funkcjonalnością wspólną dla obu klas (tj. **GraphEdge**) listy sąsiedztwa oraz macierze posiadają flagę `directed`. Decyduje ona, czy graf jest skierowany lub nieskierowany. Zmienia to działanie algorytmów, więc flaga nie powinna być zmieniania, o ile użytkownik wie, co robi (w ten sposób można łatwo przekonwertować graf nieskierowany na skierowany). Zmiana tej flagi to doprowadzić do stanów nieokreślonych, więc należy ją ustawić tylko w konstruktorze lub ew. podczas zamiany formatu grafu. Oprócz tego posiadają poniższe funkcje własne;

3.1. Lista Sąsiedztwa

Poniższe metody dotyczą list sąsiedztwa.

3.1.1. Wydrukowanie listy sąsiedztwa.

By zobaczyć na ekranie wygląd listy sąsiedztwa, wywołujemy funkcję `print(G)` na Grafie G . Warto zaznaczyć, że (w przeciwieństwie do innych bibliotek, jak NetworkX) dla list (jak i macierzy) sąsiedztw można użyć znaków UTF-8 (w tym Emoji) do opisanego grafu, jak widać na grafice 3.

```

>>> I = Graph({'🌱', '🐛', '🍌', '🐼', '🐱'}, [(('🌱', '🐛'), ('🐛', '🍌'), ('🍌', '🐼'), ('🐼', '🐱')), ('🐛', '🐼'), ('🐛', '🍌'), ('🍌', '🐼')], name="Foodchain")
{'🌱': ['🐛'], '🐛': ['🍌', '🐼'], '🍌': ['🐼'], '🐼': [], '🐱': ['🐼']}
>>> print(I)
Multigraph Foodchain with the following params:
Vertex: {'🐛', '🍌', '🐼', '🐱', '🌱'}, Adjacency list: {'🌱': ['🐛'], '🐛': ['🍌', '🐼'], '🍌': ['🐼'], '🐼': [], '🐱': ['🐼']}
>>> I.remove_vertex('🐼')
>>> print(I)
Multigraph Foodchain with the following params:
Vertex: {'🐛', '🍌', '🐱', '🌱'}, Adjacency list: {'🌱': ['🐛'], '🐛': ['🍌'], '🍌': [], '🐱': []}

```

Obraz 3: Łańcuch pokarmowy przedstawiony jako graf. Zastosowano tutaj transformację grafu typu "5 lat więzienia" (*sic!*).

3.1.2. Konwersja z/do Macierzy sąsiedztwa

Konwersja z/do macierzy została zaimplementowana poprzez tzw. wzorzec projektowy „Fabryki”. Python nie pozwala na więcej niż jeden konstruktor, więc jedynym obejściem było skorzystanie z tego wzorca projektowego. Służy do tego dekorator `@classmethod`. Wymaga to również modyfikacji konstruktora klasy. Ciekawym problemem do rozwiązania było zastosowanie zapętlających się zależności modułów; zostało to rozwiązane poprzez użycie konstrukcji `import` zamiast `from`. W ten sposób klasy mają odniesienia do siebie nawzajem.

```

1 >>> from grapyte import *
2 >>> G = Graph({'a', 'b', 'c', 'd'}, [(('a', 'b'), ('a', 'c'), ('a', 'd'), ('b', 'c'), ('c', 'd'))], name = "Lecture", directed=False)
3 {'a': ['b', 'c', 'd'], 'b': ['a', 'c'], 'c': ['a', 'b', 'd'], 'd': ['a', 'c']}
4 >>> H = G.to_GraphAdj()
5 >>> H
6 <grapyte.GraphAdj.GraphAdj object at 0x73556d47bef0>
7

```

Obraz 4: Konwersja na macierz sąsiedztwa.

3.2. Macierz Sąsiedztwa

Poniższe metody dotyczą macierzy sąsiedztwa.

3.2.1. Wydrukowanie informacji o macierzy

Za pomocą komendy `print(G)` możemy wydrukować reprezentację macierzy, wraz z podpisami wierzchołków. Wymusiło to użycie dwóch słowników odwrot-

nych, tak by szybko znaleźć przypisanie wierzchołka do jego opisu tekstowego. Dodanie i usunięcie wierzchołka wpływa na charakter tych przypisań, jak i na macierz sąsiedztw.

```
>>> I = Graph({'🐱', '🐶', '🐭', '🐹', '🐼'}, [('🐱', '🐶'), ('🐶', '🐭'), ('🐭', '🐹'), ('🐹', '🐼'), ('🐼', '🐱')])
{'🐱': ['🐶'], '🐶': ['🐭'], '🐭': ['🐹'], '🐹': ['🐼'], '🐼': ['🐱']}
>>> L = I.to_GraphAdj()
>>> print(L)
Directed Matrix Multigraph Unnamed with the following params:
Vertex: {0: '🐱', 1: '🐶', 2: '🐭', 3: '🐹', 4: '🐼'}
Matrix:
🐱,   🐶,   🐭,   🐹,   🐼
🐱 [  0  1  0  0  0]
🐶 [  0  0  1  0  1]
🐭 [  0  0  0  1  0]
🐹 [  0  0  0  0  0]
🐼 [  0  0  0  0  0]
>>> L.VertexToIndex
{'🐱': 0, '🐶': 1, '🐭': 2, '🐹': 3, '🐼': 4}
>>> L.IndexToVertex
{0: '🐱', 1: '🐶', 2: '🐭', 3: '🐹', 4: '🐼'}
>>> print(L.to_Graph())
Multigraph Unnamed with the following params:
Vertex: {'🐶', '🐭', '🐹', '🐱'}, Adjacency list: {'🐱': ['🐶'], '🐶': ['🐭', '🐼'], '🐭': ['🐹'], '🐹': ['🐼']}
```

Obraz 5: Przykład użycia klasy GraphAdj.

3.2.2. Konwersja z/do Listy sąsiedztwa

Konwersja z listy sąsiedztwa została zrealizowana analogicznie do konwersji z/do Listy (patrz: 4, 5).

3.2.3. Wczytywanie grafu o formacie Graph6/Digraph6

Klasa **GraphAdj** wspiera odczyt (i częściowo zapis) z formatu Graph6 oraz Digraph6 dla grafów o wielkości nie większej niż 64².

3.2.4. N-Marszruty

Metoda `count_nWalks` oblicza ilość marszrut stopnia N dla danej krawędzi.

3.2.5. Cykle długości 3

Metoda `count_3Cycles` oblicza ilość cykli długości 3 dla danego wierzchołka.

²Zadecydowały o tym ograniczenia funkcji NumPy `np.unpackbits`, w której nie można kontrolować "paddingu" wypakowywanych liczb, przez co odczyt graph6 i digraph6 jest realizowany sekwencyjnie. Jedynym sposobem na obejście tego byłoby napisanie własnego rozszerzenia w języku C "wycinające" nadmierne bity.

```

1  def count_edges(self)-> int:
2      """
3      Gets the amount of edges in a graph.
4      """
5      if self.directed:
6          return self.adjMatrix.sum()
7      else:
8          return self.adjMatrix.sum() // 2
9
10
11  def get_deg(self, vertex: str)-> int:
12      """
13      Gets the degree of a vertex.
14      """
15      # Loops v -> v are counted twice
16      # therefore 2*a+b+c... = sum(row) + a
17      pos = self.VertexToIndex[vertex]
18      # row sum + a
19      return self.adjMatrix[pos].sum() + self.adjMatrix[pos, pos]

```

Obraz 6: Algorytm funkcji count_edges oraz get_deg.

3.2.6. Suma krawędzi

Metoda count_edges() oblicza dla dowolnego grafu sumę krawędzi (Obraz 6).

3.2.7. Stopień wierzchołka

Metoda get_deg() na obrazie 6 oblicza stopień wierzchołka za pomocą sumy dla danego wiersza macierzy. Warto zauważyć, że funkcja liczy pętle dwukrotnie.

3.2.8. Liczba Annihilacji

Użyty algorytm dla liczby Annihilacji przedstawiono na Obrazie 7. W kodzie źródłowym w komentarzu zamieszczono szybszą, ale wygenerowaną przez LLM implementację tej funkcji.

4. Możliwości Rozwoju projektu

- Testowanie za pomocą modułu pytest zamiast prostych stringów
- Rozwinięcie funkcjonalności konwersji g6/d6 na grafy przekraczające 64 wierzchołki
- Napisanie rozszerzenia do biblioteki numpy w C/Cythonie
- Użycie algorytmów/funkcji współbieżnych zamiast sekwencyjnych (np. instrukcje wektorowe AVX)
- Zastąpienie Pythonowych funkcji biblioteką NumPy.

```

1  def get_annihilation(self)-> int:
2      """
3          Gets the annihilation number of a Graph.
4          Uses get_deg.
5          """
6      degs = sorted([self.get_deg(v) for v in self.VertexToIndex.
7          keys()])
8      edges = self.count_edges() # not using the "handshake"
9      lemma as
10     # our graphs can be both directed and undirected.
11     d_sum = 0
12     i = 0
13     while (d_sum <= edges):
14         d_sum += degs[i]
15         i += 1
16     return i - 1

```

Obraz 7: Prosty algorytm liczby Annihilacji.

- Dodanie możliwości pokolorowania grafu (potrzebne do realizacji n-cyklów).

5. Wnioski oraz spostrzeżenia

- Zaimplementowano wzorec projektowy "Fabryki" oraz "Budowniczego".
- Nauczono się parsowania formatu graph6. Napisano własnoręcznie parser graph6.
- Poznano trzeci typ grafów. Oprócz wcześniej poznanej list oraz macierzy sąsiedztwa, pierwotna implementacja zawierała listę wierzchołków. Nie był to czas zmarnowany, bo użyto tą klasę jako szkielet do napisania pozostałych klas.
- Poznano nowe własności Pythona. Mimo, że język Python nie jest autorowi obcy, poznano nowe oraz czasami niechłujne kulisy Pythona (hakowanie dekoratorami).
- Rozwiązano problem zapętłonych importów.

Prezentuje również listę spostrzeżeń, która nie jest wyczerpująca.

- Python nie ma dobrego wsparcia dla funkcji obiektowych. Największym problemem z klasami było napisanie wyspecjalizowanego konstruktora przyjmującego wyłącznie klasę listy sąsiedztwa. Dekorator `@classmethod` (odpowiadający za wzorec fabryki) wyłącznie emuluje funkcję języków obiektowych, które C++, Java czy C# wspiera wyśmienicie.
- Python bardzo słabo wspiera prawidłowe typowanie. W językach kompilowanych dosyć łatwo jest ograniczyć wartości, które może przyjąć program. Dużą część napisanych klas zajął tzw *"Typechecking"*, czyli walidacja wejść. W języku Javascript, poprzez nakładkę TypeScript można

wymusić na użytkowniku stosowanie interfejsu zgodnie z przeznaczeniem. Postanowiono (z sugestiami osób z zewnątrz), że następny nowy projekt będzie pisany w języku TypeScript.

- Drobne zmiany potrafią zmienić cały kod. Wraz ze wzrostem wielkości klasy omylnie zostały wprowadzone regresje, których rozwiązanie zajęło więcej czasu niż pisanie niektórych funkcji.
- Programowanie obiektowe zachęca do ponownego użycia kodu. Podczas pracy zauważono wzory, które powtarzały się wielokrotnie w kodzie. Część kodu można było przekopiować. W ten sposób zaimplementowano dodatkowe funkcje w krótkim czasie. Niestety ze względu na ograniczenia językowe (Python...) nie wprowadzono getterów i setterów, bo nie istnieją (pomijając dekorator `@property`).
- Pisanie kodu (samemu) uczy. Probowano zastosować sztuczną inteligencję do napisania części funkcji. Jednakże modele językowe (tutaj *Claude*) popełniają trudne do uchwycenia błędy, przez co wygenerowany przez nie kod bywa często błędny. Przełożyło się to na czas pisania kodu.
- Pisanie kodu samemu jest wolne. Problem z napisaniem dobrego programu nie jest w tym, żeby go napisać, ale żeby zaprojektować go tak, by pisanie jego było najłatwiejsze.